

OPERATOR SOP

Release, deployment, and production verification

Kindred Asterling AI Coaching

A practical workflow from branch creation to production proof

Repository: Kindred-Asterling-AI-Coaching

Document date: 27 August 2026

Production health endpoint: <https://kindred-asterling-ai-coaching.com/api/healthz>

Definition of done: the exact commit is locally validated, the protected-branch merge gate

is green, Coolify has deployed the intended image, and both application and database health

checks plus a representative user flow have been verified.

How to use this SOP

Follow the sections in order. Record the commit SHA, pipeline URL, Coolify deployment, image digest if available, database backup identifier, and production verification time in the release record.

Repository reality check: this checkout contains GitHub Actions under `.github/workflows/`

and no committed `.gitlab-ci.yml`. The GitLab pipeline section below is the required target

pattern if GitLab is the release system; do not claim GitLab proof until that file and its pipeline

exist.

Kindred Asterling AI Coaching | Internal release operations

1. Release Controls

Production branch	<code>main</code> is protected. Changes land through a reviewed pull/merge request; do not push directly.
Release unit	The immutable Git commit SHA and the resulting container image. Never deploy an unrecorded local build.
Application image	The repository-root <code>Dockerfile</code> , built by Coolify. CI validation and production deployment are separate concerns.
Data changes	Back up PostgreSQL first. Schema initialization/migration is deliberately not part of container startup.
Secrets	Use local environment tooling or Coolify secrets. Never commit real keys, database URLs, signing secrets, or encryption values.

Release record

Change / ticket	—
Commit SHA	—
Reviewer / approval	—
Pipeline URL	—

Coolify deployment / image digest	
	—
Backup identifier	
	—

2. Branch and Local Validation

1. Start from an up-to-date `main` and create a focused branch.

`git fetch origin`

`git switch main`

Use a branch name that states the change. Keep unrelated edits out of the branch. Before committing, inspect the diff and the status.

`git status --short`

Required local checks

The project uses Node 24 and pnpm 10.28.1 in the production Dockerfile. Run the same lockfile-safe install and validation commands used by CI.

`corepack enable`

`corepack prepare pnpm@10.28.1 --activate`

`pnpm install --frozen-lockfile`

`pnpm run typecheck`

`pnpm --filter @workspace/api-server test`

`pnpm run build`

For API changes that require a database, use a disposable PostgreSQL instance and set `DATABASE_URL`. Do not point local migration or push commands at production.

Commit and push

`git add path/to/changed-files`

Prefer SSH remotes for developer machines. For HTTPS, use a GitLab token as the password; `write_repository` grants Git-over-HTTP push access and does not authenticate the GitLab API.

3. Pipeline and Merge Gates

Current checkout: the committed workflow is GitHub Actions at `.github/workflows/ci.yml`. It runs on

pushes to main and pull requests targeting main, provisions PostgreSQL 17, installs with pnpm, pushes

the test schema, runs typecheck/tests, and builds production artifacts.

Observed GitHub Actions gate

Open the pull request and wait for the CI workflow to complete. Confirm the PostgreSQL service is healthy and all steps pass: install, database schema push, typecheck, API tests, and production build. Do not merge if the build is red, if the lockfile changes unexpectedly, or if a security finding has not been reviewed.

GitLab target gate

If the repository is moved to GitLab CI, use three stages and keep security scans in the test stage:

stages:

- build
- test
- deploy

include:

- template: Jobs/SAST.gitlab-ci.yml
- template: Jobs/Secret-Detection.gitlab-ci.yml
- template: Jobs/Dependency-Scanning.v2.gitlab-ci.yml

The `test` stage must exist. Dependency Scanning v2 is the current SBOM-based template; the legacy dependency template should not be introduced for a new pipeline. Configure protected `main` to require a successful pipeline, review approval, and no unresolved blocking security finding.

CI isolation rules

- Each job runs in an isolated environment. A Docker daemon or installed dependency tree is not automatically available in the next job.
- Use `cache` for pnpm download content; use `artifacts` to pass explicit build outputs between stages.
- If using Docker-in-Docker, push the image from the same build job or pass an explicit registry reference. Do not assume Docker layers persist across jobs.

4. Review, Merge, and Promotion

1. Open the pull/merge request with a concise summary, test evidence, migration impact, environment-variable impact, and rollback plan.
2. Have a reviewer inspect security-sensitive changes, authentication, payment/webhook handling, database schema changes, and public API behavior.
3. Confirm the exact commit that passed validation is the commit being merged. Re-run the gate after conflict resolution.
4. Merge using the repository's protected-branch policy. Record the resulting `main` SHA.

Before deployment: verify a PostgreSQL backup exists, migration direction is understood, required

Coolify variables are present, and the release is backward-compatible with the currently running

application.

5. Coolify Deployment

Coolify builds the root Dockerfile independently of CI. Configure the application as follows:

Repository / branch	The private repository and production branch containing the approved commit
Build pack	Dockerfile
Dockerfile path	<code>/Dockerfile</code>
Application port	8080
Public build arguments	<code>VITE_CLERK_PUBLISHABLE_KEY</code> , <code>VITE_SENTRY_DSN</code>
Runtime secrets	Database, Clerk server, webhook, encryption, email, AI-provider, and optional integration values from <code>.env.example</code>
Liveness check	<code>GET /api/healthz</code> expects HTTP 200
Database readiness	<code>GET /api/healthz/db</code> expects HTTP 200; failure is HTTP 503

The image runs as the unprivileged `kindred` user and includes a Docker health check for `/api/healthz`. Do not mount a persistent volume over `/app`. Persist PostgreSQL data and Ollama models only.

Schema operation

- Empty new database: run `pnpm --filter @workspace/db initialize` from a trusted admin environment.
- Existing database: back up, run `pnpm --filter @workspace/db migrate`, then review and run `pnpm --filter @workspace/db push`.
- Never make startup mutate production schema automatically.

Push-to-deploy

Use Coolify's Git integration or deploy webhook for the production branch. Restrict webhook access, avoid exposing secrets in URLs, and confirm the deployment log identifies the expected commit. If a GitLab integration is used, grant only the repository access required to clone the private repository.

6. Production Verification

1. Wait for the Coolify deployment to report healthy and confirm the deployed commit/image digest.
2. Check application liveness:

```
curl --fail --silent --show-error https://kindred-asterling-ai-coaching.com/api/healthz
```

```
curl --fail --silent --show-error https://kindred-asterling-ai-coaching.com/api/healthz/db
```

3. Confirm the response is HTTP 200 and the expected JSON. Liveness proves the API accepts requests; the DB endpoint proves PostgreSQL connectivity.
4. Test Clerk login, one representative coaching/Ollama response, and one relevant user journey.
5. Confirm Clerk and payment webhook provider logs show 2xx delivery. Confirm Google Calendar callback configuration if that integration is enabled.
6. Check application logs for startup errors, database errors, authentication failures, and unexpected restart loops.

Cloudflare

Use Full (strict) TLS, do not cache `/api/*`, preserve webhook request bodies, and enable WebSockets if required. If Super Bot Fight Mode blocks the health monitor, create a WAF Custom Rule matching the exact host and path and skip the `http_request_sbfm` phase. This does not bypass Bot Fight Mode.

7. Rollback and Incident Handling

1. Stop promotion and record the failing deployment, commit SHA, symptoms, and time.
2. For an application-only failure, redeploy the preceding successful Coolify deployment or immutable image digest.
3. Re-run both health checks and the representative user flow.
4. Do not automatically reverse a database migration. Use a reviewed down-migration or restore the pre-deploy backup only after stopping writes and accepting the documented data-loss window.
5. Keep at least two known-good application images and their configuration available.

On replacement, the API handles SIGTERM by stopping its scheduler, draining the HTTP listener, and closing the PostgreSQL pool. Allow at least 30 seconds before forced termination.

8. Definition of Done

- Branch was created from current protected main.
- `pnpm install --frozen-lockfile`, typecheck, tests, and production build passed.
- Pull/merge request received required review and all required CI checks passed.
- Exact approved commit SHA was merged and recorded.
- PostgreSQL backup was confirmed before promotion.
- Coolify used the root Dockerfile, port 8080, and correct public build arguments.
- Runtime secrets were configured without committing sensitive values.
- Coolify deployment and image/commit identity were confirmed.
- `/api/healthz` and `/api/healthz/db` returned HTTP 200.
- Login, one coaching response, and relevant webhook deliveries were verified.
- Logs were reviewed and rollback target was identified.

Owner: engineering / operations | Review this SOP when the CI provider, deployment platform, or production contract

changes.